ONLINE EXAM SYSTEM

Project Features, Implementation, Request Flow and Database Guide

Java 17 • JavaFX • Maven • JDBC • MySQL

Prepared for practical project demonstration and DBMS evaluation

1. Project Overview

The Online Exam System is a Java desktop application built with JavaFX and connected to a MySQL relational database using JDBC. It is designed to manage multiple-choice examinations digitally. Students and administrators have separate workflows. Students can register and log in, attempt available timed exams, submit answers, view scores and check result history. Administrators can log in separately, manage exams and questions, and review results.

1.1 Problem the project solves

Manual examinations require paper, manual answer checking and time-consuming result preparation. This project centralizes exam information, supports computer-based attempts and can automate scoring for objective questions.

1.2 Main objectives

- Separate student and administrator access.
- Manage exam and question records in a database.
- Provide a timed multiple-choice exam interface.
- Calculate and store results for later review.
- Use relational database keys to keep related records connected.
- Demonstrate JavaFX UI development, Java logic, JDBC connectivity and SQL operations.

2. Technologies Used and Their Purpose

Technology	How it is used
Java 17	Main programming language. Implements classes, methods, validations and application logic.
JavaFX	Creates the desktop screens: text fields, password fields, buttons, labels, tables, dashboards, exam screen and timer.
Maven	Build tool and dependency manager. Reads pom.xml, compiles code and runs the JavaFX application using the configured plugin.
JDBC	Java Database Connectivity API. Allows Java code to connect to MySQL, execute SQL and read returned rows.
MySQL Server	Stores student/admin accounts, exams, questions and results in tables.
MySQL Workbench	Lets the developer inspect schemas, run SQL commands, check data and review table constraints.
VS Code	Used to edit Java, CSS, Maven configuration and run commands in the integrated terminal.
JavaFX CSS (if configured)	Controls screen appearance such as spacing, colours, button styles and text formatting.

3. Project Structure and Responsibility

Layer / package	Responsibility
ui	Screens and user interactions. Examples discussed include Login, AdminLogin, StudentDashboard, ExamScreen, ExamManagement, QuestionManagement and ResultReports.
service	Business logic and coordination between UI and data access. Examples include ExamService, AuthenticationService and ResultService.
dao	Data Access Objects that hold SQL operations and map database rows to Java objects. AdminDAO and ResultDAO

	have been discussed.
model	Java classes that represent data, such as Student, Admin, Exam, Question and Result.
util	Shared utilities, especially DatabaseConnection for opening a JDBC connection.
resources/css	Optional JavaFX stylesheet location, commonly src/main/resources/css/app.css when loaded as a classpath resource.
pom.xml	Maven project configuration, including Java version, JavaFX and MySQL JDBC dependencies, and the run plugin.

3.1 Why separate these layers?

The UI should handle display and user input; services should handle rules and coordinate work; DAO/JDBC code should handle database queries; models should represent data. This separation makes code easier to read, debug and maintain.

4. Main Features and How They Work

4.1 Student registration

1. The student enters name, email, password and phone number in the registration form.
2. The UI checks required fields are not blank.
3. The UI calls a service method, such as registerStudent(...).
4. The service executes an INSERT statement using a JDBC PreparedStatement.
5. MySQL stores the new row in students. The application shows a success or failure message.
6. Email should be unique in the database to prevent duplicate accounts.

4.2 Student login

7. The student enters the registered email and password.
8. The Login screen calls ExamService.login(email, password).
9. The service queries students for a matching account.
10. If a matching record is returned, the Student object is passed to StudentDashboard; otherwise the UI displays an invalid-credentials message.
11. If login fails, verify that registration saved to the same database and that the password storage format matches the login logic.

Security note: the previously discussed ExamService code stored and compared passwords as plain text. This is not secure for real deployment. A production-ready implementation should store salted password hashes and verify them using an established password-hashing library.

4.3 Administrator login and registration

Administrators use a separate login screen and dashboard. The discussed AdminDAO login query checks that the email/password match and that is_active = 1. Therefore, a newly registered admin may need activation before logging in. The project requirement discussed earlier restricts admin email addresses to the @onlineexam.com domain; confirm the current registration code enforces this.

4.4 Exam management

An administrator can create and manage exam details such as title, description, duration and total marks. The exam is stored in exams. The proposed database design places admin_id in exams so the application can identify which administrator created each exam. If the application includes an availability/published field, it can control which exams students can attempt.

4.5 Question management

An administrator adds multiple-choice questions to a selected exam. A question can store the question text, options, correct option and marks. The exam_id foreign key connects each question to its parent exam. One exam can contain multiple questions.

4.6 Timed exam attempt

12. The student dashboard lists exams available to the student.
13. The student opens an exam and the exam screen displays questions and answer options.
14. A countdown timer indicates the remaining time.
15. The student chooses answers and submits. Whether time expiry automatically submits depends on the implementation in the current code.
16. The application compares submitted options with correct options and calculates marks.

4.7 Results and result history

After submission, the application saves a result associated with the student and exam. The student can view past results. An administrator can review result reports. SQL JOINs can combine results with students and exams so the report displays names and exam titles rather than only numeric IDs.

4.8 UI styling

JavaFX controls and layout classes create the screen structure. CSS can style the controls and provide consistent colours, spacing and typography. If UiStyle.createScene(...) loads /css/app.css, the stylesheet should normally be in src/main/resources/css/app.css so Maven includes it on the runtime classpath.

5. Database Design and Relationships

The application uses a relational database. Each table stores one category of information. A primary key (PK) uniquely identifies a row; a foreign key (FK) references a related row in another table.

Table	Key fields and purpose in the proposed design
students	PK student_id; name, email, password, phone. Stores student account details.
admins	PK admin_id; name, email, password, is_active. Stores admin accounts and activation state.
exams	PK exam_id; FK admin_id; title, description, duration, total_marks, availability/status, created_at. Stores exams and the creator admin.
questions	PK question_id; FK exam_id; question_text, option_a, option_b, option_c, option_d, correct_option, marks. Stores questions for an exam.
results	PK result_id; FK student_id and FK exam_id; score, total_marks, attempted_at. Stores exam attempt results.

5.1 Relationship explanations

Relationship	Meaning
admins → exams (1:N)	One administrator can create many exams. Each exam points to the creator through exams.admin_id.
exams → questions (1:N)	One exam contains many questions; each question belongs to one exam.
students → results (1:N)	One student can have multiple result records over time.
exams → results (1:N)	One exam can have results from multiple student attempts.

The table and column names above reflect the proposed logical design discussed for this project. Verify them against the live schema. The ER diagram alone does not create foreign-key constraints; those constraints must exist in MySQL and the exam-creation code must store the logged-in admin's ID.

5.2 How the database supports the workflow

- Registration inserts a row into students or the admin account table.
- Exam creation inserts an exam row and records the creator's admin_id.
- Question creation inserts question rows with the selected exam_id.
- Exam submission inserts a result row with student_id and exam_id plus score information.
- Reports read joined data from related tables.

6. Request Flow: What Happens After a User Clicks a Button?

```
Example: Student clicks Login
1. Login.java reads the email and password fields.
2. The UI calls ExamService.login(email, password).
3. ExamService prepares a parameterized SQL SELECT statement.
4. DatabaseConnection.getConnection() connects Java to MySQL.
5. PreparedStatement sends the query to MySQL.
6. ResultSet contains the matching row, if one exists.
7. ExamService maps the row into a Student object.
8. Login.java opens StudentDashboard, or displays an error if no row matches.
```

6.1 Generic request flow

```
User action in JavaFX UI
        ↓
Input validation in the screen
        ↓
Service method (business rules)
        ↓
DAO / JDBC PreparedStatement
        ↓
DatabaseConnection
        ↓
MySQL executes SQL
        ↓
Rows or update count returned
        ↓
Java object / status returned to UI
        ↓
Dashboard, table or status message updated
```

6.2 Insert, read, update and delete

Operation	SQL purpose	Project example
INSERT	Adds a new row.	Register student, create exam, add question, save result.
SELECT	Reads existing rows.	Login, list available exams, load questions, show history.
UPDATE	Changes an existing row.	Edit exam/question, change availability or activate an admin, if supported.
DELETE	Removes a row.	Delete an exam or question, if supported; consider related foreign-key records.

The exact features available in each screen depend on the current code. Demonstrate only the operations that are implemented and working.

7. Project Flow from Start to Finish

```
ADMIN WORKFLOW
Admin opens Admin Login
        ↓
Credentials checked; account must be active if required
        ↓
Admin Dashboard
        ↓
Create / manage an exam (exam row stores admin_id)
        ↓
Add questions (each question stores exam_id)
        ↓
Make exam available to students, if the feature exists
        ↓
Review result reports after students submit

STUDENT WORKFLOW
Student registers
        ↓
Student logs in
        ↓
Student Dashboard
        ↓
Select an available exam
        ↓
Read questions and choose answers while timer runs
        ↓
Submit answers
        ↓
Application calculates score
        ↓
Result stored with student_id and exam_id
        ↓
Student views result/history
```

7.1 Overall data flow

The admin creates the exam first, then adds questions under that exam. The student selects an available exam and submits answers. The result links the student and exam, so later the application can show student history and admin reports. This is the core end-to-end flow of the project.

8. How to Run the Project

17. Start MySQL Server and open MySQL Workbench.
18. Check the database name and ensure the required tables exist.
19. Open the project root in VS Code—the folder containing pom.xml.
20. Open Terminal → New Terminal in VS Code.
21. Run java -version to check Java; this project context uses Java 17.
22. Run mvn -version to check Maven; the discussed setup uses Maven 3.9.8.
23. Check DatabaseConnection.java for the correct JDBC URL, schema name, username and password. Do not share credentials publicly.
24. Run mvn clean javafx:run from the project root.

25. Wait for Maven to compile and launch the JavaFX login window.
26. Use test accounts and records to demonstrate student and admin flows.

8.1 Useful commands

Command	Use
java -version	Checks installed Java version.
mvn -version	Checks Maven version and the Java Maven uses.
mvn clean javafx:run	Cleans, compiles and runs the configured JavaFX app; run in the folder with pom.xml.
mvn clean compile	Cleans and compiles without launching the UI.
SHOW DATABASES;	Lists MySQL databases.
USE online_exam;	Selects the schema, if its name is online_exam.
SHOW TABLES;	Lists tables in the selected schema.
DESCRIBE students;	Shows columns and data types; repeat for each table.
SHOW CREATE TABLE exams;	Shows table definition and constraints, including foreign keys.
SELECT DATABASE();	Shows the currently selected MySQL schema.
SELECT student_id, name, email FROM students;	Checks whether student records exist without displaying password values.
SELECT admin_id, name, email, is_active FROM admins;	Checks admin records and activation state without displaying password values.

Replace online_exam with the actual database name used by your application.

8.2 Database inspection checklist

```
SELECT DATABASE();
SHOW TABLES;
DESCRIBE students;
DESCRIBE admins;
DESCRIBE exams;
DESCRIBE questions;
DESCRIBE results;

SHOW CREATE TABLE exams;
SHOW CREATE TABLE questions;
SHOW CREATE TABLE results;
```

9. Common Problems and Troubleshooting

Problem	Checks to perform
Invalid student login	Confirm registration succeeded, the row exists in the same schema, email is correct and password format matches the login query.
Invalid admin login	Confirm the account exists and is_active = 1 if the DAO query requires it.
Access denied for MySQL	Check username, password and database privileges in DatabaseConnection.
Unknown database	Check the schema name in Workbench and the JDBC URL.
Table not found	Run SELECT DATABASE() and SHOW TABLES; verify the spelling.
Foreign-key error	Check that the parent row exists, column types match and the referenced column is a primary/unique key.
No exams visible	Confirm an exam exists and its availability matches the query used by the student screen.
JavaFX dependency/module error	Check Java version, pom.xml dependencies/plugin, source paths and terminal location.
CSS does not appear	Confirm stylesheet path and that UiStyle adds it to the scene; classpath CSS usually belongs in

	src/main/resources/css/.

10. Short Project Explanation for Evaluation

“My project is an Online Exam System built using Java 17, JavaFX, Maven, JDBC and MySQL. It has separate student and administrator workflows. Students can register, log in, attempt available multiple-choice exams with a timer, submit answers and review results. Administrators can manage exams and questions and view result reports. The database separates students, admins, exams, questions and results into related tables. The exams table stores admin_id to identify the administrator who created each exam, questions stores exam_id to connect each question to its exam, and results stores student_id and exam_id to connect an attempt to a student and exam. In the application, the JavaFX UI calls service methods, JDBC executes parameterized SQL against MySQL, and the returned data is displayed on the relevant screen.”

11. Final Evaluation Checklist

- ☐ Application launches with mvn clean javafx:run from the folder containing pom.xml.
- ☐ MySQL Server is running and DatabaseConnection uses the correct schema.
- ☐ The ERD matches the actual table columns and constraints.
- ☐ The exams table has admin_id and a real FK if claiming creator tracking is implemented.
- ☐ Student registration and login work with a test account.
- ☐ Admin login works with an active test admin.
- ☐ At least one exam and its questions exist.
- ☐ A test attempt can be submitted and its result can be viewed.
- ☐ Demonstrate only features that currently work.
- ☐ Avoid showing passwords or connection credentials.

Accuracy note

This guide is based on the project structure and code discussed in the conversation, not a fresh inspection of the complete source repository or live MySQL schema. Confirm exact columns, foreign-key constraints, timer behaviour, score calculation, and optional availability/edit/delete features in the current implementation before claiming them as implemented.